

Shell Scripting for Linux Administrators

Foundations, Syntax, Real-World Use Cases, and a Complete Production-Grade Example

A block-by-block reference: everything from the shebang line to a full server health-check and backup automation script.

1. What Is a Shell Script?

A shell script is a plain text file containing a sequence of commands that the shell (bash, sh, zsh, etc.) reads and executes line by line, just as if you had typed them yourself at the terminal. Shell scripting lets a Linux administrator automate repetitive tasks such as backups, log rotation, user provisioning, service monitoring, and deployment workflows without manual intervention.

Shell scripts are interpreted, not compiled. This means the shell reads and executes the script on the fly, one line at a time, which makes them fast to write and test but slower to run than compiled programs — a good trade-off for automation and glue logic.

2. What You Need Before You Start

2.1 Prerequisites / Tools

- A Linux/Unix environment with a shell installed (bash is the de facto standard).
- A text editor: vim, nano, or VS Code — anything that saves plain text.
- Execute permission on the script file (chmod +x).
- Basic familiarity with the command line: navigating directories, redirection, pipes.
- Know which shell you are targeting — bash, sh (POSIX), dash — since syntax and features differ.

2.2 The Shebang Line

Every script should start with a shebang (#!) that tells the OS which interpreter to use. This must be the very first line of the file.

`example.sh`

```
#!/bin/bash
# This script prints a greeting
echo "Hello, Linux Administrator!"
```

2.3 Making a Script Executable

```
chmod +x example.sh    # give execute permission
./example.sh           # run it (current directory)
bash example.sh        # or run explicitly with bash, no chmod needed
```

2.4 Checking Syntax

```
bash -n script.sh      # syntax check only, does not execute
shellcheck script.sh   # static analysis tool - highly recommended
```

3. Core Building Blocks

3.1 Comments

```
# This is a single line comment
: '
This is a
multi-line comment block
'
```

3.2 Variables

Variables hold no data type in bash — everything is treated as a string unless used in an arithmetic context. There is no space around the equals sign.

```
name="Md"
count=10
readonly PI=3.14          # constant, cannot be reassigned
unset count               # delete a variable

echo "Hello, $name"
echo "Hello, ${name}!"    # braces avoid ambiguity
echo "Count is: $count"
```

3.3 Special / Positional Variables

```
$0      # script name
$1 $2   # first, second argument passed to the script
$#      # number of arguments
$@      # all arguments as separate words
$*      # all arguments as a single string
$?      # exit status of the last command (0 = success)
$$      # PID of the current script
$!      # PID of the last background job
```

3.4 Command Substitution & Arithmetic

```
today=$(date +%F)          # preferred syntax
files=`ls /var/log`        # legacy backtick syntax (avoid)

sum=$((5 + 3))
echo "Sum: $sum"

let count++
((count += 5))
```

3.5 Quoting Rules

```
echo "Double quotes allow $variable expansion"
echo 'Single quotes are literal - $variable stays as text'
echo "Path: \"$HOME\""      # escape a literal quote
```

3.6 Reading User Input

```
read -p "Enter your name: " uname
```

```
echo "Welcome, $uname"

read -s -p "Enter password: " pass # -s hides input
```

4. Conditions (Decision Making)

4.1 if / elif / else

```
#!/bin/bash
disk_usage=$(df / | awk 'NR==2 {print $5}' | tr -d '%')

if [ "$disk_usage" -ge 90 ]; then
    echo "CRITICAL: disk usage at ${disk_usage}%"
elif [ "$disk_usage" -ge 75 ]; then
    echo "WARNING: disk usage at ${disk_usage}%"
else
    echo "OK: disk usage at ${disk_usage}%"
fi
```

4.2 Test Operators — [] vs [[]]

- [] is the POSIX *test* command — portable, works in sh/dash/bash.
- [[]] is a bash keyword — supports pattern matching, regex (==), and && / || inside.
- (()) is for arithmetic comparisons only, e.g. ((a > b)).

```
# Numeric comparisons
[ "$a" -eq "$b" ] # equal
[ "$a" -ne "$b" ] # not equal
[ "$a" -gt "$b" ] # greater than
[ "$a" -lt "$b" ] # less than
[ "$a" -ge "$b" ] # greater or equal
[ "$a" -le "$b" ] # less or equal

# String comparisons
[ "$str1" = "$str2" ] # equal
[ "$str1" != "$str2" ] # not equal
[ -z "$str1" ] # true if string is empty
[ -n "$str1" ] # true if string is NOT empty

# File tests
[ -e "$file" ] # exists
[ -f "$file" ] # is a regular file
[ -d "$dir" ] # is a directory
[ -r "$file" ] # is readable
[ -w "$file" ] # is writable
[ -x "$file" ] # is executable
[ -s "$file" ] # exists and is not empty

# bash-only extended test
if [[ "$hostname" =~ ^web[0-9]+$ ]]; then
```

```
    echo "Matches web server naming pattern"
fi
```

4.3 Logical Operators

```
[ -f file.txt ] && echo "exists"          # AND: run right side if left
succeeds
[ -f file.txt ] || echo "missing"         # OR: run right side if left fails
if [ "$a" -gt 0 ] && [ "$b" -gt 0 ]; then echo "both positive"; fi
```

4.4 case Statement

```
#!/bin/bash
read -p "Enter service action (start|stop|restart|status): " action

case "$action" in
    start)
        systemctl start nginx
        ;;
    stop)
        systemctl stop nginx
        ;;
    restart)
        systemctl restart nginx
        ;;
    status)
        systemctl status nginx
        ;;
    *)
        echo "Usage: $0 {start|stop|restart|status}"
        exit 1
        ;;
esac
```

5. Loops

5.1 for Loop

```
# Loop over a list
for user in alice bob carol; do
    echo "Creating user: $user"
    useradd "$user"
done

# C-style loop
for (( i=1; i<=5; i++ )); do
    echo "Iteration $i"
done

# Loop over files
```

```
for file in /var/log/*.log; do
    echo "Log file: $file"
done
```

```
# Loop over command output
for pid in $(pgrep nginx); do
    echo "nginx PID: $pid"
done
```

5.2 while Loop

```
counter=1
while [ "$counter" -le 5 ]; do
    echo "Count: $counter"
    ((counter++))
done
```

```
# Read a file line by line - the correct, safe way
while IFS= read -r line; do
    echo "Line: $line"
done < /etc/hosts
```

```
# Infinite monitoring loop
while true; do
    uptime
    sleep 60
done
```

5.3 until Loop

```
count=1
until [ "$count" -gt 5 ]; do
    echo "Attempt $count"
    ((count++))
done
```

5.4 break and continue

```
for i in {1..10}; do
    if [ "$i" -eq 3 ]; then
        continue      # skip this iteration
    fi
    if [ "$i" -eq 8 ]; then
        break          # exit the loop entirely
    fi
    echo "$i"
done
```

6. Functions

Functions group reusable logic, accept arguments the same way scripts do (\$1, \$2 ...), and can return an exit status via return (0-255) or emit data via echo/printf that the caller captures with command substitution.

```
log() {
    local level="$1"
    local message="$2"
    echo "$(date '+%F %T') [$level] $message" >> /var/log/myscript.log
}

is_service_running() {
    local svc="$1"
    systemctl is-active --quiet "$svc"
    return $?      # 0 = running, non-zero = not running
}

log "INFO" "Script started"

if is_service_running "sshd"; then
    log "INFO" "sshd is running"
else
    log "ERROR" "sshd is DOWN"
fi
```

7. I/O Redirection, Pipes, Arrays, and Error Handling

7.1 Redirection

```
command > file.txt      # stdout to file (overwrite)
command >> file.txt     # stdout to file (append)
command 2> error.log    # stderr to file
command > out.log 2>&1  # stdout AND stderr to same file
command &> all.log       # bash shortcut for the above
command < input.txt     # read stdin from a file
command 2>/dev/null     # discard errors
```

7.2 Pipes and Common Filters

```
ps aux | grep nginx | grep -v grep
cat access.log | awk '{print $1}' | sort | uniq -c | sort -nr | head -10
df -h | grep -v tmpfs
journalctl -u sshd --since "1 hour ago" | tail -50
```

7.3 Arrays

```
servers=("web01" "web02" "db01")
echo "${servers[0]}"      # first element
echo "${servers[@]}"      # all elements
echo "${#servers[@]}"     # array length

for s in "${servers[@]"; do
    ping -c 1 "$s" &> /dev/null && echo "$s is UP" || echo "$s is DOWN"
```

```
done
```

```
declare -A capacity          # associative array
capacity[web01]="80%"
capacity[db01]="45%"
echo "${capacity[web01]}"
```

7.4 Error Handling and Safety Flags

```
set -e          # exit immediately if a command exits non-zero
set -u          # error on use of an unset variable
set -o pipefail # a pipeline fails if ANY command in it fails
set -x          # print each command before executing (debugging)
```

```
trap 'echo "Error on line $LINENO"; exit 1' ERR
trap 'echo "Cleaning up temp files"; rm -f /tmp/myscript.*' EXIT
```

7.5 Exit Codes

```
exit 0    # success
exit 1    # general error
exit 2    # misuse of shell command
# convention: use distinct non-zero codes for distinct failure reasons
```

8. Essential Commands a Linux Administrator Uses in Scripts

Category	Commands	Typical Use
Files & text	cat, less, head, tail, cut, sort, uniq, wc	View/inspect/count text data
Search	grep, egrep, find, locate	Pattern search, file discovery
Text processing	sed, awk, tr	Transform, substitute, extract fields
Processes	ps, top, kill, pkill, pgrep, nice	Inspect/manage running processes
Services	systemctl, service, journalctl	Start/stop/inspect services & logs
Disk & FS	df, du, mount, lsblk, fdisk	Storage inspection and management
Networking	ip, ping, curl, wget, ss, netstat, scp, rsync	Connectivity, transfer, diagnostics
Users & perms	useradd, usermod, passwd, chmod, chown, chgrp	Account and permission management
Scheduling	cron, at, systemd timers	Run scripts on a schedule
Archiving	tar, gzip, zip, rsync	Backup and compression
Package mgmt	apt, yum, dnf, rpm	Install/update/remove software

9. Real-World Use Case Scenarios

- **Automated backups** — tar/rsync a directory or database nightly to local or remote storage, with rotation of old backups.
- **Log rotation & cleanup** — compress or delete logs older than N days to free disk space.
- **Service health monitoring** — periodically check that critical services (nginx, sshd, docker) are running and restart or alert if not.
- **User provisioning** — bulk create/delete users and groups from a CSV list during onboarding/offboarding.
- **Deployment automation** — pull latest code, run migrations, restart services, roll back on failure.
- **System hardening / auditing** — check for open ports, weak permissions, expired passwords, and report findings.
- **Disk & resource alerts** — warn via email/Slack when CPU, memory, or disk usage crosses a threshold.
- **Cron-driven ETL glue** — move/transform files between directories, trigger downstream jobs.
- **Bulk remote operations** — SSH into a list of servers to run the same command (patching, restarts) via a for loop.

10. Real-World Complex Example

The script below is a realistic production-style tool a Linux administrator might actually deploy via cron. It combines everything covered above: functions, variables, arrays, conditions, loops, case, redirection, logging, argument parsing, error handling with trap/set, and exit codes. It performs three real jobs in one tool: a system health check, a directory backup with rotation, and an email-style alert (printed to log, easily wired to mail/curl for Slack).

server_maintenance.sh — full script

```
#!/bin/bash
#####
# server_maintenance.sh
# Purpose : Health check + backup + alerting, meant to run nightly via cron
# Usage   : ./server_maintenance.sh [--backup-dir DIR] [--threshold PCT]
#####

set -uo pipefail # unset-var errors + pipeline failures are caught
                  # (set -e intentionally omitted so we can handle errors
                  ourselves)

# ----- Configuration -----
LOG_FILE="/var/log/server_maintenance.log"
BACKUP_SRC="/etc"
BACKUP_DEST="/var/backups/etc"
RETENTION_DAYS=7
DISK_THRESHOLD=80
SERVICES=("sshd" "nginx" "cron")
ALERT_EMAIL="ops-team@example.com"

# ----- Functions -----
log() {
    local level="$1"; shift
    local msg="$*"
    echo "$(date '+%F %T') [$level] $msg" | tee -a "$LOG_FILE"
}

send_alert() {
    local subject="$1"
    local body="$2"
    # In production: mail -s "$subject" "$ALERT_EMAIL" <<< "$body"
    log "ALERT" "$subject :: $body"
}

usage() {
    echo "Usage: $0 [--backup-dir DIR] [--threshold PCT]"
    exit 2
}

check_disk_usage() {
```

```

log "INFO" "Checking disk usage (threshold ${DISK_THRESHOLD}%)"
while read -r line; do
    usage_pct=$(echo "$line" | awk '{print $5}' | tr -d '%')
    mount_point=$(echo "$line" | awk '{print $6}')

    if [ "$usage_pct" -ge "$DISK_THRESHOLD" ]; then
        send_alert "Disk usage high" "$mount_point at ${usage_pct}%"
    else
        log "INFO" "$mount_point OK at ${usage_pct}%"
    fi
done < <(df -h --output=pcent,target | tail -n +2)
}

check_services() {
    log "INFO" "Checking critical services: ${SERVICES[*]}"
    local failed=0
    for svc in "${SERVICES[@]"; do
        if systemctl is-active --quiet "$svc"; then
            log "INFO" "$svc is running"
        else
            log "ERROR" "$svc is NOT running - attempting restart"
            if systemctl restart "$svc"; then
                log "INFO" "$svc restarted successfully"
            else
                send_alert "Service failure" "$svc failed to restart"
                ((failed++))
            fi
        fi
    done
    return "$failed"
}

run_backup() {
    local timestamp
    timestamp=$(date +%Y%m%d_%H%M%S)
    local archive="${BACKUP_DEST}/etc_backup_${timestamp}.tar.gz"

    mkdir -p "$BACKUP_DEST"

    log "INFO" "Starting backup of $BACKUP_SRC -> $archive"
    if tar -czf "$archive" "$BACKUP_SRC" 2>>"$LOG_FILE"; then
        log "INFO" "Backup completed: $archive ($(du -h "$archive" | cut
            -f1))"
    else
        send_alert "Backup failed" "tar exited with an error, check $LOG_FILE"
        return 1
    fi

    log "INFO" "Rotating backups older than $RETENTION_DAYS days"
    find "$BACKUP_DEST" -name "etc_backup_*.tar.gz" -mtime
        "+${RETENTION_DAYS}" -print -delete \
        | while IFS= read -r old; do

```

```

        log "INFO" "Deleted old backup: $old"
    done
}

cleanup() {
    log "INFO" "Maintenance run finished with exit code $?"
}
trap cleanup EXIT
trap 'log "ERROR" "Unexpected failure near line $LINENO"; exit 1' ERR

# ----- Argument parsing
-----
while [ "$#" -gt 0 ]; do
    case "$1" in
        --backup-dir)
            BACKUP_DEST="$2"
            shift 2
            ;;
        --threshold)
            DISK_THRESHOLD="$2"
            shift 2
            ;;
        -h|--help)
            usage
            ;;
        *)
            echo "Unknown option: $1"
            usage
            ;;
    esac
done

# ----- Main
-----
log "INFO" "=== Starting server_maintenance.sh (PID $$) ==="

check_disk_usage

check_services
service_failures=$?
if [ "$service_failures" -gt 0 ]; then
    log "WARN" "$service_failures service(s) failed to recover"
fi

run_backup

log "INFO" "=== Maintenance run complete ==="
exit 0

```

10.1 What This Example Demonstrates

- **Shebang + safety flags** — `set -uo` pipefail catches unset variables and broken pipelines.

- **Configuration block** — variables and an array (SERVICES) grouped for easy editing.
- **Functions** — log(), send_alert(), check_disk_usage(), check_services(), run_backup(), each with a single responsibility.
- **Loops** — a while/read loop parsing df output, a for loop over the SERVICES array, and a while/read loop consuming find's output for rotation.
- **Conditions** — if/else for thresholds, service state, and tar success/failure; a case statement for argument parsing.
- **Process substitution** — <(...) lets a while loop read df output without spawning a subshell that would hide variable updates.
- **trap** — EXIT and ERR traps guarantee logging even when something fails midway.
- **Real commands an admin actually uses** — df, systemctl, tar, find, du, mkdir, all chained naturally.
- **Exit codes and cron-friendliness** — clear exit 0 / non-zero, and a single log file cron can also mail on failure.

10.2 Scheduling It

```
# crontab -e
0 2 * * * /usr/local/bin/server_maintenance.sh --threshold 85 >>
/var/log/cron_maintenance.log 2>&1
```

11. Quick Reference Cheat Sheet

Concept	Syntax
Shebang	<code>#!/bin/bash</code>
Variable	<code>name="value"</code> (no spaces around =)
Use variable	<code>\$name</code> or <code>\${name}</code>
Command substitution	<code>result=\$(command)</code>
Arithmetic	<code>((a + b))</code> or <code>=\$((a + b))</code>
If	<code>if [cond]; then ... elif [cond]; then ... else ... fi</code>
Case	<code>case \$var in pattern) cmds;; esac</code>
For loop	<code>for i in list; do ... done</code>
C-style for	<code>for ((i=0;i<N;i++)); do ... done</code>
While loop	<code>while [cond]; do ... done</code>
Until loop	<code>until [cond]; do ... done</code>
Function	<code>name() { ... ; return N; }</code>
Array	<code>arr=(a b c)</code> <code>\${arr[@]}</code> <code>\${#arr[@]}</code>
Redirect stdout	<code>cmd > file</code> <code>cmd >> file</code>
Redirect stderr	<code>cmd 2> file</code> <code>cmd &> file</code>
Pipe	<code>cmd1 cmd2</code>
Exit status	<code>\$?</code> (0 = success)
Safety flags	<code>set -euo pipefail</code>
Trap	<code>trap 'cmds' EXIT ERR INT</code>